

Take-Home Assignment: Bus Charging Scheduler

Time: 3–4 days **Stack:** Python + Streamlit. One repo, one process. **Submission:** Hosted link + GitHub repo (via the form at the end)

Stack and hosting

Use Python and Streamlit. Everything — scheduling logic, scenario loading, UI — lives in one Python repo, one process.

- **Host on [Streamlit Community Cloud](#)** — free, 2 clicks from GitHub
- **Libraries:** Any Python library that installs via pip works fine. Streamlit Community Cloud reads your `requirements.txt` and installs everything automatically.

We're not testing UI skills or DevOps here. Streamlit removes the friction so you can focus on the actual problem.

The problem

Electric buses run on a fixed route with 4 charging stations along the way.

Route: Bengaluru → A → B → C → D → Kochi

Buses travel in both directions — some go Bengaluru → Kochi, others go Kochi → Bengaluru — and share the same charging stations.

Each bus starts its trip with a **full charge** — Bengaluru and Kochi have slow chargers that fully charge buses before they depart, so you can assume every bus leaves its origin with a 240 km range. **Only A, B, C, and D are scheduling charging stations** — the endpoints are not part of the scheduling problem.

Stations exist so buses can recharge along the way. **Which stations a bus uses is up to your scheduler**, as long as the bus never runs out of range between charges.

Each station has **1 charger**, so when multiple buses want to charge at the same station around the same time, the scheduler has to decide who goes first and who waits.

Your job: build the scheduler that decides each bus's charging plan and the order in which buses use the chargers.

The rules

Physical constants

- **Battery range:** 240 km on a full charge
- **Charging:** always to full, takes 25 minutes (fixed)
- **All buses travel at the same speed** (no traffic, no variation)

The route

Segment	Distance
Bengaluru → A	100 km
A → B	120 km
B → C	100 km
C → D	120 km
D → Kochi	100 km
Total	540 km

Travel time is determined by distance (use a consistent speed in your simulation — e.g. 60 km/h means a 100 km segment takes 100 minutes).

Buses

- **20 buses total per scenario** — 10 going Bengaluru→Kochi, 10 going Kochi→Bengaluru
- Each bus has a scheduled **departure time** from its starting end (Bengaluru or Kochi)
- Each bus belongs to one of 3 **operators**: KPN, Freshbus, Flixbus

Charging plans

A bus can drive a maximum of **240 km on a full charge**. Charging always fills the battery back to full.

This means between any two consecutive charges — or between starting and the first charge, or between the last charge and arrival — a bus cannot travel more than 240 km. If it would, the schedule is invalid.

This means a bus going Bengaluru → Kochi cannot complete the trip without charging at least **2 times** (total trip is 540 km). The scheduler chooses *which* 2 (or more) stations the bus uses.

Hard rules that must always hold

- One bus per charger at a time (1 charger per station)
 - Charging is always exactly 25 minutes
 - A bus must never run out of range between two consecutive charges (or between segments without a charge)
 - A bus visits stations in route order — no backtracking
-

What to optimize for

When the scheduler has flexibility (which stations to use, who charges first), it should weigh three **soft rules**:

1. **Individual bus** — no single bus should wait too long
2. **Operator** — each operator's fleet should run smoothly as a group
3. **Overall** — total time across the whole network should be low

These weights should be **tunable** — engineers will change them as we learn what matters operationally. **Don't hardcode them.**

The one thing we really care about

Your scheduler must be built to scale.

The scenarios below are small (20 buses, 1 charger per station, 3 operators). The real world won't stay that small. Over the coming months we'll be tweaking weights based on field analysis — what *actually* matters operationally only becomes clear once we run real buses. We'll also keep adding rules as we learn more (priority buses, time-of-day electricity costs, driver shifts, multiple routes sharing stations, etc.).

Your underlying scheduling framework needs to handle this gracefully:

- **Changing a weight** must be trivial — a value in one obvious place, not scattered across code
- **Adding a new rule** must not require rewriting the engine — just defining the new rule
- **Growing the world** (more buses, stations, operators, routes) must not need a rewrite

Picking the right approach is part of the assignment. Solve the problem in a way that doesn't paint you into a corner.

Designing your data structure (important)

Below we're giving you only the **departure schedules** for 5 scenarios — just bus IDs, operators, directions, and departure times.

A scenario IS your data structure. When we say "the 5 scenarios," we mean 5 data files — in whatever format you design — that fully describe each situation for your scheduler to read. The schedule tables below are just the human-readable input; you decide what the file actually looks like, what fields it carries, and how the rest of the world (route, stations, weights, etc.) is represented.

You design the actual data structure your scheduler and UI use. What fields do you include? How do you represent direction, route, stations, weights, the per-bus charging plan output? That's your call — and one of the strongest signals to us about how well you understood the problem.

Think like the designer, not the order-taker

The product team has given you today's requirements: 4 stations, 1 charger each, 3 operators, 20 buses, fixed route. **Don't just build that.**

If you only build exactly what was asked, you'll be rewriting half your system the moment anything changes. Strong engineers anticipate. Before you write any code, sit with the problem and ask: *what will the next ask probably look like? And the one after that?* The world rarely stays the way the first spec describes it.

We're judging you on this directly. Write down, in your ARCHITECTURE.md, the **full set of changes you anticipated** when designing your data structure — and how your design handles each of them without code changes. Be specific. The breadth and quality of this list is one of the strongest signals to us about how you think.

The harder version of this question: *if I told you tomorrow that something in this world is different, what's the chance your code breaks?* A good data structure makes that chance close to zero for anything reasonable. A bad one makes it close to one.

Think hard about what your scheduler actually needs as input and what makes sense as output. A well-designed data model makes everything downstream easier; a sloppy one creates bugs you'll spend hours chasing.

The 5 scenarios

Below are the departure schedules for each scenario. Encode these into your own data format and ship all 5 with your submission.

All scenarios use:

- Route: Bengaluru → A → B → C → D → Kochi (segments: 100, 120, 100, 120, 100 km)
- Battery range: 240 km
- Charging time: 25 min, to full
- 1 charger per station
- Default weights: **individual = 1.0, operator = 1.0, overall = 1.0** — **except where noted**

Scenario 1 — Even spacing

Buses depart every 15 minutes in each direction starting 19:00. Baseline case.

Bus ID	Operator	Direction	Departure
bus-BK-01	kpn	Bengaluru→Kochi	19:00
bus-BK-02	freshbus	Bengaluru→Kochi	19:15
bus-BK-03	flixbus	Bengaluru→Kochi	19:30
bus-BK-04	kpn	Bengaluru→Kochi	19:45
bus-BK-05	freshbus	Bengaluru→Kochi	20:00
bus-BK-06	flixbus	Bengaluru→Kochi	20:15
bus-BK-07	kpn	Bengaluru→Kochi	20:30
bus-BK-08	freshbus	Bengaluru→Kochi	20:45
bus-BK-09	flixbus	Bengaluru→Kochi	21:00
bus-BK-10	kpn	Bengaluru→Kochi	21:15
bus-KB-01	freshbus	Kochi→Bengaluru	19:00
bus-KB-02	flixbus	Kochi→Bengaluru	19:15
bus-KB-03	kpn	Kochi→Bengaluru	19:30
bus-KB-04	freshbus	Kochi→Bengaluru	19:45
bus-KB-05	flixbus	Kochi→Bengaluru	20:00
bus-KB-06	kpn	Kochi→Bengaluru	20:15
bus-KB-07	freshbus	Kochi→Bengaluru	20:30
bus-KB-08	flixbus	Kochi→Bengaluru	20:45
bus-KB-09	kpn	Kochi→Bengaluru	21:00

Bus ID	Operator	Direction	Departure
bus-KB-10	freshbus	Kochi→Bengaluru	21:15

Scenario 2 — Bunched start

Buses from both directions depart in a tight cluster (every 8 min) over the first 50 minutes, then space out. Creates heavy early contention.

Bus ID	Operator	Direction	Departure
bus-BK-01	kpn	Bengaluru→Kochi	19:00
bus-BK-02	freshbus	Bengaluru→Kochi	19:08
bus-BK-03	flixbus	Bengaluru→Kochi	19:16
bus-BK-04	kpn	Bengaluru→Kochi	19:24
bus-BK-05	freshbus	Bengaluru→Kochi	19:32
bus-BK-06	flixbus	Bengaluru→Kochi	19:40
bus-BK-07	kpn	Bengaluru→Kochi	19:48
bus-BK-08	freshbus	Bengaluru→Kochi	20:03
bus-BK-09	flixbus	Bengaluru→Kochi	20:18
bus-BK-10	kpn	Bengaluru→Kochi	20:33
bus-KB-01	freshbus	Kochi→Bengaluru	19:00
bus-KB-02	flixbus	Kochi→Bengaluru	19:08
bus-KB-03	kpn	Kochi→Bengaluru	19:16
bus-KB-04	freshbus	Kochi→Bengaluru	19:24
bus-KB-05	flixbus	Kochi→Bengaluru	19:32
bus-KB-06	kpn	Kochi→Bengaluru	19:40
bus-KB-07	freshbus	Kochi→Bengaluru	19:48
bus-KB-08	flixbus	Kochi→Bengaluru	20:03
bus-KB-09	kpn	Kochi→Bengaluru	20:18
bus-KB-10	freshbus	Kochi→Bengaluru	20:33

Scenario 3 — Asymmetric load

10 buses going Bengaluru→Kochi (15 min spacing), only 4 going Kochi→Bengaluru. Tests how the scheduler handles uneven traffic across directions.

Bus ID	Operator	Direction	Departure
bus-BK-01	kpn	Bengaluru→Kochi	19:00
bus-BK-02	freshbus	Bengaluru→Kochi	19:15
bus-BK-03	flixbus	Bengaluru→Kochi	19:30
bus-BK-04	kpn	Bengaluru→Kochi	19:45
bus-BK-05	freshbus	Bengaluru→Kochi	20:00
bus-BK-06	flixbus	Bengaluru→Kochi	20:15
bus-BK-07	kpn	Bengaluru→Kochi	20:30
bus-BK-08	freshbus	Bengaluru→Kochi	20:45
bus-BK-09	flixbus	Bengaluru→Kochi	21:00
bus-BK-10	kpn	Bengaluru→Kochi	21:15
bus-KB-01	freshbus	Kochi→Bengaluru	19:00
bus-KB-02	flixbus	Kochi→Bengaluru	19:35
bus-KB-03	kpn	Kochi→Bengaluru	20:10
bus-KB-04	freshbus	Kochi→Bengaluru	20:45

Scenario 4 — Operator-heavy

One operator (KPN) dominates the Bengaluru→Kochi fleet (8 of 10 buses). Tuning the "operator" weight up vs down should produce visibly different schedules.

Weights for this scenario: `individual = 1.0, operator = 2.0, overall = 1.0`

Bus ID	Operator	Direction	Departure
bus-BK-01	kpn	Bengaluru→Kochi	19:00
bus-BK-02	kpn	Bengaluru→Kochi	19:15
bus-BK-03	kpn	Bengaluru→Kochi	19:30
bus-BK-04	kpn	Bengaluru→Kochi	19:45

Bus ID	Operator	Direction	Departure
bus-BK-05	kpn	Bengaluru→Kochi	20:00
bus-BK-06	kpn	Bengaluru→Kochi	20:15
bus-BK-07	kpn	Bengaluru→Kochi	20:30
bus-BK-08	kpn	Bengaluru→Kochi	20:45
bus-BK-09	freshbus	Bengaluru→Kochi	21:00
bus-BK-10	flibus	Bengaluru→Kochi	21:15
bus-KB-01	freshbus	Kochi→Bengaluru	19:00
bus-KB-02	flibus	Kochi→Bengaluru	19:15
bus-KB-03	kpn	Kochi→Bengaluru	19:30
bus-KB-04	freshbus	Kochi→Bengaluru	19:45
bus-KB-05	flibus	Kochi→Bengaluru	20:00
bus-KB-06	kpn	Kochi→Bengaluru	20:15
bus-KB-07	freshbus	Kochi→Bengaluru	20:30
bus-KB-08	flibus	Kochi→Bengaluru	20:45
bus-KB-09	kpn	Kochi→Bengaluru	21:00
bus-KB-10	freshbus	Kochi→Bengaluru	21:15

Scenario 5 — Worst case convergence

All 20 buses dispatched within a 72-minute window (every 8 min) from both ends. By the time buses reach inner stations (B and C), they collide. Maximum contention.

Bus ID	Operator	Direction	Departure
bus-BK-01	kpn	Bengaluru→Kochi	19:00
bus-BK-02	freshbus	Bengaluru→Kochi	19:08
bus-BK-03	flibus	Bengaluru→Kochi	19:16
bus-BK-04	kpn	Bengaluru→Kochi	19:24
bus-BK-05	freshbus	Bengaluru→Kochi	19:32

Bus ID	Operator	Direction	Departure
bus-BK-06	flibus	Bengaluru→Kochi	19:40
bus-BK-07	kpn	Bengaluru→Kochi	19:48
bus-BK-08	freshbus	Bengaluru→Kochi	19:56
bus-BK-09	flibus	Bengaluru→Kochi	20:04
bus-BK-10	kpn	Bengaluru→Kochi	20:12
bus-KB-01	freshbus	Kochi→Bengaluru	19:00
bus-KB-02	flibus	Kochi→Bengaluru	19:08
bus-KB-03	kpn	Kochi→Bengaluru	19:16
bus-KB-04	freshbus	Kochi→Bengaluru	19:24
bus-KB-05	flibus	Kochi→Bengaluru	19:32
bus-KB-06	kpn	Kochi→Bengaluru	19:40
bus-KB-07	freshbus	Kochi→Bengaluru	19:48
bus-KB-08	flibus	Kochi→Bengaluru	19:56
bus-KB-09	kpn	Kochi→Bengaluru	20:04
bus-KB-10	freshbus	Kochi→Bengaluru	20:12

What to build

A single Python + Streamlit app that:

The scheduler

- Reads any scenario from your data files
- Uses your framework, with weights from the scenario
- Decides each bus's **charging plan** (which stations it uses) and the **order** in which buses use each station
- Computes, for each bus, the timeline: when it charges where, how long it waits, when it arrives at Kochi or Bengaluru

The UI

- A **dropdown** at the top to pick a scenario

- A **scenario view** showing the input (raw data or readable table) so reviewers can see what's being fed in
- A **per-bus timetable** — for each bus, show its full timeline: charging stations used, time at each, wait (if any), final arrival
- A **per-station view** — for each of A, B, C, D, show the order in which buses charged there

That's it. No metrics dashboards, no maps, no animations. Pick a scenario → see the input → see what the scheduler decided.

How we'll test your submission

When we open your hosted link, we'll:

1. **Open the app** — land on it and see the scenario dropdown immediately
 2. **Pick a scenario** — say, Scenario 1
 3. **See the scenario data** displayed so we can see the input
 4. **Look at the per-bus timetable** — for each bus, is the plan sensible? Did the bus charge enough times to make the trip? Did the wait times look reasonable?
 5. **Look at the per-station view** — does the order at each station make sense given the weights?
 6. **Cycle through all 5 scenarios** — every one should produce a sensible, defensible result
-

Making your own calls

You'll find gaps in this spec — that's intentional. **Make your own assumptions, document them, and move forward.** Don't email us with clarifying questions about edge cases or modeling decisions — we want to see how you handle ambiguity. If you assume something we wouldn't have, you can defend it in the interview.

(The only thing worth emailing about is genuine technical blockers — hosting issues, broken access, etc.)

Deliverables

1. Hosted link

Working web app with all 5 scenarios in the dropdown.

2. GitHub repo (must be public)

- All code
 - All 5 scenarios encoded in your data format
 - **README.md** — how to run it locally, how to change a weight, how to add a new rule
 - **ARCHITECTURE.md** — explain:
 - What framework / approach you chose for the scheduler, and why it's the right fit for this problem
 - Your data structure design
 - **The list of future changes you anticipated** when designing the data structure, and for each one, how your design handles it without code changes
 - How you'd change a weight (with a code example)
 - How you'd add a new rule (with a code example)
 - The assumptions you made
-

Tips

- **Use AI tools freely.** We do too. Just be ready to explain every decision in the interview.
 - **Don't over-engineer.** No auth, no DB, no maps. In-memory state is fine.
 - **Design your data structure first.** It will shape everything else.
-

How we'll evaluate

Area	What we're looking for
Approach	Did you pick a scheduling approach that's the right fit? Can you defend why?
Scalability	Adding a new rule is genuinely small (we'll test this live). The engine doesn't need a rewrite when the world grows.
Weight tunability	Changing a weight is one obvious value in one obvious place — not scattered through code
Data modeling & foresight	Clean structure that captures inputs and outputs. Did you anticipate how the world might change and design for it — without being told what to anticipate? The breadth of your anticipated changes (and how cleanly your design handles them) is a key signal.

Area	What we're looking for
Correctness	Schedules respect the 240 km range rule; different weights → different (defensible) schedules
Code quality	Clear and easy to extend
Docs	Honest about what's done, what's not, what's next

Submission

 **Deadline: June 2nd**

Submit via this form: <https://forms.gle/51xrFoUeGj9PD6KQA>

You'll be asked for:

- Hosted Streamlit app URL (must be public)
- GitHub repo URL (must be public)
- The approach / framework you used for scheduling
- A few brief notes about your build

If you spot an error after submitting, just submit the form again with the corrected info — we'll use your latest submission.

What happens next

After review, we'll schedule a technical round where you'll:

1. **Walk us through your solution** — demo a couple of scenarios live, including your data structure design and your framework choice
2. **Run a scenario we hand you on the spot** — we'll give you a fresh departure schedule and you'll encode it
3. **Extend the data without rewriting** — we'll throw a curveball: add a new station, double the chargers somewhere, swap in a new operator, change a segment distance. You should be able to do it through data alone.
4. **Defend your architecture** — your framework choice, your data model, why your approach scales
5. **Add a new rule live** — we'll ask you to add a new soft or hard rule to the scheduler and see how cleanly it slots in

We're hiring someone who can think clearly about ambiguous problems, ship working systems quickly, and own their decisions. Good luck!